# Git, GitHub, and VS Code

## Agentic AI for Research: From One Task to Orchestration

Tomas Larroucau

Arizona State University

Monday, October 26, 2026

# Why This Workshop?

**The Problem:**

- Research projects grow messy fast
- Code, writing, and decisions drift apart
- More agents can mean more work to supervise

**What We Will Do:**

- Work in one research environment
- Turn a task into a reviewable change
- Coordinate agents with clear ownership

**By the end**

You can give an agent a bounded task, inspect its changes, and decide whether the work is ready to share.

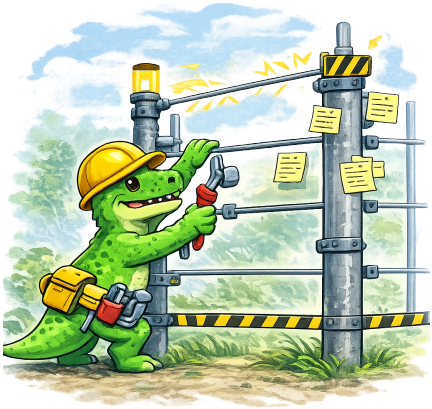
# Our 90 Minutes

| Minutes | What we do |
|---|---|
| 0–7 | The research workflow and today's goal |
| 7–18 | VS Code and the project template |
| 18–31 | Git, issues, and a reviewable change |
| 31–43 | Claude Code and Codex: ten useful comparisons |
| 43–58 | Live demo: plan, edit, inspect, verify |
| 58–68 | Short exercise or paired review |
| 68–82 | Orchestration: tasks, sessions, worktrees, resources |
| 82–90 | Questions and one next step |

**Keep moving**

If setup fails, follow the demonstration or work with a partner. The appendix is for later.

# The Symbiosis: Agents as Builders



- **VS Code**: read, write, run, and review in the project
- **Git and GitHub**: saved history and shared decisions
- **Agents**: help build and check the workflow itself

The dinosaur can help build the fence. We still inspect the fence.

Git can restore recorded files. It cannot retract a message, recover every unsaved file, or undo an external action.

# VS Code: An Integrated Research Environment

**Read and Write**

- Browse papers, notes, and project files
- Edit LaTeX and Markdown
- Preview a PDF beside its source
- Search the whole project

**Run and Collaborate**

- Run scripts in the terminal
- Inspect Git diffs and history
- Work with issues and pull requests
- Use agent extensions or CLI sessions

**Demo:** Open the project folder. Read the README, preview the slides, inspect a diff, and open an agent session.

PDF, language, and agent features depend on the extensions you install. Start with what your project needs.

# Reproducible Research Template

- `data/raw/`: teaching data, kept unchanged
- `scripts/`: processing and analysis
- `output/`: generated figures and tables
- `tex/`: paper and Beamer sources
- `Makefile`: build steps
- `README.md`: how to use the project
- `AGENTS.md`: project guidance
- `.gitignore`: untracked files to exclude

**Key Principle:** Generate research outputs from scripts. Review source edits and record the decisions behind them.

Template: `https://github.com/tlarroucau/AI_workshop`

Build commands in the appendix

# Workshop Prerequisites

**Follow and Review**

- A browser and the workshop materials
- VS Code and Git for the local exercise
- A local copy of the template
- A partner if setup is incomplete

**Run an Agent Yourself**

- One supported agent, signed in
- Check your account's access and limits
- GitHub account for hosted collaboration
- Python and LaTeX only for full builds

**Today's exercise**

A README edit works without Python, LaTeX, or an MCP server. Claude Code and Codex are demonstrated; you do not need both subscriptions.

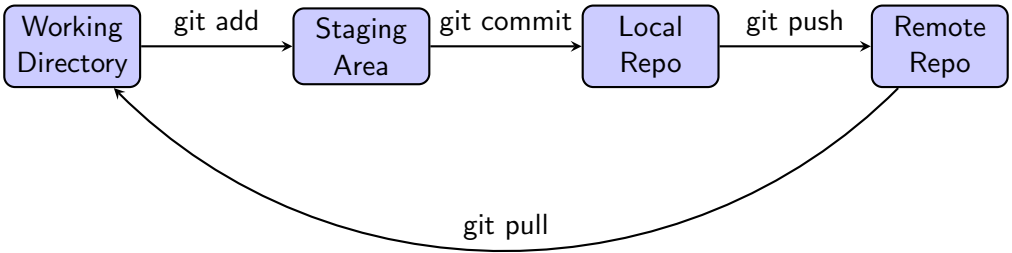
# Git Core Concepts

1. **Repository (repo)**: Project folder tracked by Git
2. **Commit**: Snapshot of the changes you staged
3. **Branch**: Parallel version of your repository
4. **Merge**: Combine changes from different branches
5. **Remote**: Another repository (often hosted on GitHub)

## Basic Workflow

```
git add file.py          # Stage changes
git commit -m "message"  # Save snapshot
git push                 # Upload to remote
```

# The Git Workflow




- Edit files in working directory
- Stage changes you want to commit
- Commit records the staged changes in local history
- Push shares with collaborators

# GitHub Issues: Shared Research Context

**An issue records the task**

- Research goal and why it matters
- Relevant files and decisions
- Owner and allowed scope
- Evidence required to finish

**Example: clarify the Quick Start**

Separate a README-only exercise from a full analysis build. Verify every listed command against the Makefile. Keep analysis and data unchanged.

Coauthors can read the issue after the chat ends. Ask each new session to read it and report what remains unresolved.

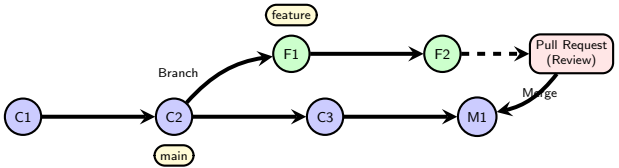
The example is a task brief. Creating or updating a hosted issue requires the appropriate access and authorization.

# GitHub Workflow: Pull Requests

**What are Pull Requests (PRs)?**

- Propose changes to repository
- Enable code review before merging
- Discuss implementation details

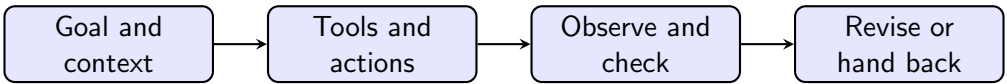
**PR Workflow:**




1. Create and push feature branch
2. Open Pull Request on GitHub
3. Review, discuss, and fix
4. Merge when approved

**Review:** Does the diff satisfy the issue? What evidence supports merging?

# What Is Agentic AI?




- An agent can read files, edit, run commands, and use connected tools when permitted.
- The model, its tools, context, and permissions jointly determine what it can do.
- Completion is a claim to check against the requested output and evidence.

# Model, Agent, and Interface

## Three different choices

**Model**: the system generating responses.

**Agent harness**: instructions, tools, permissions, and execution loop.

**Interface**: terminal, editor, desktop app, or web session.

Choosing a Claude model inside Copilot still uses Copilot's agent harness. It does not turn that session into Claude Code.

**Compare workflows on the same task:** context used, control of edits, review quality, time, and allowance consumed.

VS Code: agent harnesses

# Claude Code and Codex: Context and Planning

| Feature / use | Claude Code | Codex |
|---|---|---|
| **1. Instructions** Project conventions | `CLAUDE.md`; can import `@AGENTS.md`. Recent versions also support an `AGENTS.md` fallback. | `AGENTS.md`, including guidance scoped to subdirectories. |
| **2. Skills** Repeat a workflow | `SKILL.md` in `.claude/skills/`; invoke the named skill. | `SKILL.md` in `.agents/skills/`; use `/skills` or `$name`. |
| **3. Plan first** Discuss before editing | `--permission-mode plan` or Shift+Tab. | `/plan` in supported sessions; read-only permissions add an explicit edit boundary. |

**Research use:** store project conventions once; make a table-audit skill; agree the scope of a revision before editing.

Claude instructions, skills, planning; Codex instructions, skills, planning.

# Claude Code and Codex: Output, Tools, Control

| Feature / use | Claude Code | Codex |
|---|---|---|
| **4. Output style** Match the audience | Named styles; try `/output-style explanatory` while learning. | Instructions and `/personality` where supported; these are not identical to Claude's style system. |
| **5. Tools / MCP** Connect the workflow | Built-in tools plus configured MCP servers. | Built-in tools plus configured MCP servers. |
| **6. Permissions** Bound the actions | Permission modes and shell sandboxing are separate controls. | Approval policy and sandbox settings are separate controls. |

**Research use:** request a short instructor explanation, connect only needed tools, and inspect what can read, edit, execute, or send data.

Claude styles, MCP, permissions; Codex commands, MCP, security.

# Claude Code and Codex: Sessions and Remote Work

| Feature / use | Claude Code | Codex |
|---|---|---|
| **7. Sessions** Resume a task | `--continue`, `--resume`, or `/resume`. | `codex resume`; `/compact` manages long context. Naming and forks depend on the surface. |
| **8. Remote control** Supervise work elsewhere | Remote Control connects to a session on a running host. Eligibility and admin settings apply. | Remote connections are surface-dependent. CLI remote control is experimental; check the installed version. |

Save research decisions in files or issues. A resumed chat is useful context, but collaborators need a durable record.

Remote control is a view into work on another host, not automatic migration to cloud execution. Rehearse this optional demo in advance.

Claude sessions, remote control; Codex commands, remote connections.

# Claude Code and Codex: Delegation and Review

| Feature / use | Claude Code | Codex |
|---|---|---|
| **9. Delegation** Separate bounded tasks | Subagents use separate contexts. `--worktree NAME` isolates a working directory. | Subagents and Git worktree workflows are supported; setup depends on the surface. |
| **10. Review** Challenge the change | Ask a separate session or reviewer agent to inspect the task, diff, and checks. | `/review`; connected GitHub review is a separate integration. |

**Research use:** one agent drafts, another checks sources, and a reviewer looks for mistakes. Assign each writer distinct files or a worktree.

Claude's experimental `/advisor` consults another model; it is different from Plan Mode. Availability and extra usage depend on provider and billing.

Claude subagents, advisor; Codex subagents, worktrees, review.

# Live Demo: One Task, Visible Evidence

**Task:** Make the README's Quick Start match how the repository actually builds.

1. Trace one existing paper table to its source; read the task brief and Makefile.
2. Ask for a plan. State scope and the acceptance criteria.
3. Make the edit in an isolated branch or output copy.
4. Read the diff and check the commands against the project.
5. Ask an independent session to find errors or omissions.
6. Show a prepared issue/PR example and explain verified closeout.

Fallback: use the prepared before/after diff if authentication or the live agent stalls. We can inspect a complete change without waiting for a long run.

# A Prompt You Can Inspect

## Plan first

Read README.md, AGENTS.md, and Makefile. Explain how someone can do a documentation exercise and how they can run a full build. List inconsistencies and propose the smallest README edit. Do not change files yet.

## Then implement the agreed scope

Edit README.md only. Keep research code and data unchanged. Verify each command against the repository. Return the diff, checks you actually performed, and unresolved questions. Stop before any push or hosted action.

A prompt expresses intent. Permission settings and a bounded workspace provide additional control.

# Ten-Minute Exercise: Review a Change

1. **2 minutes**: Read the task and define what a correct result must include.
2. **4 minutes**: Ask one agent for a README edit, or inspect the prepared diff with a partner.
3. **3 minutes**: Find one claim to verify. Compare it with the Makefile or source file.
4. **1 minute**: Report: ready, needs changes, or blocked. Give one reason.

**Success**

You can explain what changed, what was checked, and what still needs a human decision.

# You Choose the Level of Engagement

| **You specify** | **What the agent can help produce** |
|---|---|
| Research question | Alternatives, assumptions, evidence to collect |
| Project design | Task breakdown, dependencies, risks, acceptance criteria |
| Bounded task | A proposed change and evidence that it works |
| Exact implementation | Specific edits, commands, or code you want executed |

Move between levels as needed. Keep ownership of research choices and the decision to accept or share results.

# Parallel Work Needs Distinct Ownership

| Session | Task and output | Scope | Status |
|---|---|---|---|
| Planner | Agree scope and task brief | Read-only | Ready |
| Claude Code A | Draft Quick Start edit | README; worktree A | Working |
| Codex B | Check build instructions | Read-only; findings | Working |
| Reviewer C | Review candidate diff | Read-only; findings | Waiting for A |

Each task has one owner, a defined input, an expected output, and a stop condition. The labels here are an example, not a model ranking.

**Control:** Inspect progress, interrupt, narrow scope, or reassign. Parallelize independent work; review follows a candidate change.

# One Machine: Clearly Labeled Sessions

**A simple setup**

- Open separate terminal or editor sessions
- Label provider, task, and working folder
- Keep one shared task brief or board
- Give each writer an isolated worktree

**Optional shared view**

- VS Code documents discovery of local external Claude Code and Codex sessions
- Enable the sessions list's *External* filter
- Check provider support before continuing a session

Independent labeled sessions are enough. Local session discovery does not establish discovery of remote SSH or tmux sessions.

VS Code: manage sessions. Codex continuation has a single-writer requirement; see appendix.

# Separate Worktrees for Separate Writers

**Branch**

- A line of Git history
- Switching branches changes the current checkout

**Worktree**

- A separate working directory
- Lets writers use different branches at once

## Coordination still matters

Agree which files each agent owns. Keep outputs separate, review overlapping edits, and integrate one change at a time. Worktrees isolate files; they do not isolate shared databases or external services.

Commands are in the appendix. Start from an agreed commit and preserve existing work before branching.

# The Orchestrator Keeps the Big Picture

**Personal assistant / orchestrator**

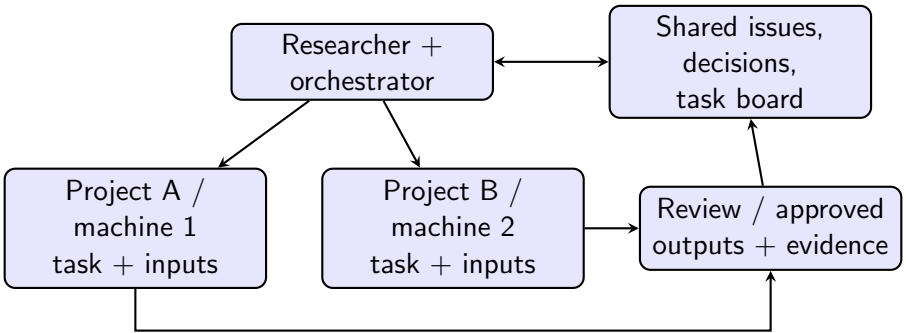
- Research goals and project priorities
- Decisions, dependencies, and open questions
- Task board and resource allocation
- Reports back in terms of outcomes

**Implementation sessions**

- A bounded task and local instructions
- Relevant files and tools
- Changes, checks, and evidence
- A concise handoff when done or blocked

Record the shared decisions outside an implementation-heavy terminal chat. Give collaborators the issue, candidate diff, and verification evidence.

# Several Projects and Machines




**Command Center demo:** a private illustration of this workflow. This diagram uses generic projects and machines.

Track input versions, execution location, allowance, and output destination. Share only the artifacts and evidence collaborators need.

# Allocate Models and Attention by Task

- Use a lower-cost available option for a bounded, easy-to-check task.
- Reserve scarce allowance and deeper reasoning for hard design choices, ambiguous failures, and consequential review.
- Stop duplicate work. Reuse a concise task brief and record useful results.
- Give reviewers the task, diff, and evidence. Ask them to find mistakes independently.

**Keep the evidence local to the claim**

"It worked on this task" is an observation. "This model is always best" requires evidence we do not have here.

# Finish the Task, Then Verify Closeout

1. Compare the result with the issue's acceptance criteria.
2. Inspect the diff, checks, and rendered outputs where relevant.
3. Resolve reviewer findings or record what remains open.
4. When sharing is authorized, link the PR to the issue and obtain review.
5. After integration, verify the intended result and the issue's status.

**A useful handoff**

What changed; where the output is; what was checked; what is blocked; which decision is needed next.

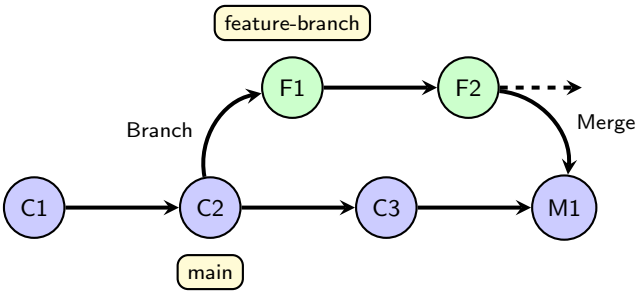
# Next Steps and Questions

1. Choose one small task in your research project.
2. Write its scope and acceptance criteria.
3. Use one agent, inspect the diff, and verify the result.
4. Add parallel sessions when the tasks can be separated clearly.

**Discussion:** Which part of your workflow would benefit most? What would you need to check before trusting the result?

Template: `https://github.com/tlarroucau/AI_workshop`
Build commands, worktree examples, and official documentation follow.

# Visualizing Branches




- **Branch**: Create a parallel timeline (`git checkout -b`)
- **Commit**: Work on the feature branch (F1, F2)
- **Merge**: Combine feature history back into main (`git merge`)
- *Note: The branch remains open after merging unless deleted*

# Essential Git Commands

```
git status                    # Check current state
git diff                      # Review unstaged edits
git diff --staged             # Review staged edits
git switch -c docs-quickstart  # Create a task branch
git add README.md             # Stage the intended file
git commit -m "Clarify Quick Start"
git log --oneline             # View saved history
git fetch                    # Retrieve remote history
```

Inspect local work before switching or integrating branches. Pull behavior depends on configuration. Push and PR creation share work with the remote repository.

# Worktree Commands: One Writer per Folder

```
# From the repository, start at an agreed commit:
git status
git worktree add ../workshop-docs -b docs-quickstart HEAD
git worktree list
# Start the writing agent inside ../workshop-docs
# Give the reviewer the candidate diff and task brief.
```

- Uncommitted edits are not copied into a new worktree.
- Use distinct output paths when sessions run concurrently.
- Preserve and integrate needed work before removing a worktree.

Git: worktree

# Git Best Practices

## Commit Messages

- Be descriptive: “Clarify README build instructions”
- Keep each commit focused on one task
- Link to the relevant issue when there is one

## What to Commit

- Source code, documentation, and small permitted teaching data
- Generated outputs only when the project’s sharing policy calls for them
- Keep credentials and restricted data outside the repository

# The .gitignore File

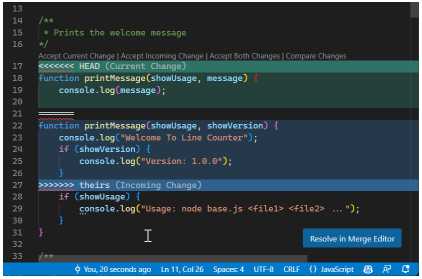
**Example patterns**

```
__pycache__/
.venv/
*.aux
*.log
output/
.env
```

- Applies to untracked files
- Does not remove a file already tracked
- Can appear in subdirectories
- Does not erase secrets from history

Inspect `git status` and the staged diff before committing. An ignore rule helps prevent accidents; it does not enforce data policy.

# Resolving Merge Conflicts in VS Code




- VS Code shows **conflict markers**: `<<<<<<< HEAD` (yours) vs `>>>>>>> theirs`
- One-click options: *Accept Current*, *Accept Incoming*, *Accept Both*, or *Compare*
- Use **Resolve in Merge Editor** for a side-by-side 3-way diff

# Useful VS Code Features

**Navigate and Review**

- Command Palette: Ctrl/Cmd+Shift+P
- Quick file open: Ctrl/Cmd+P
- Search files, symbols, and text
- Source Control: staged and unstaged diffs

**Add What You Need**

- LaTeX Workshop: source and PDF
- Python / Jupyter: scripts and notebooks
- GitHub Pull Requests and Issues
- Your chosen agent's extension

Open the project folder, select the correct environment, and use workspace settings for project-specific tools. Check Keyboard Shortcuts for platform differences.

# MCP: Model Context Protocol

- MCP connects an agent application to tools and resources exposed by a server.
- A GitHub connection can expose repository operations, depending on the server and permissions.
- Native tools and command-line tools are also valid integration paths.
- Inspect the requested permissions and configure only the access the task needs.

**For this workshop:** the instructor can use a prepared connection. Participants do not need to configure MCP for the README exercise.

MCP architecture; provider setup links in the source notes.

# Data Privacy and Permissions

- Use public or permitted teaching material for the exercises.
- Check institutional policy and the account's terms before providing research data.
- Local execution does not by itself mean local model inference.
- Inspect connected tools, network access, and output destinations.
- Ask before actions outside the task's agreed scope, including publishing or sending messages.

An offline workflow requires checking the whole configuration. A locally installed editor or agent alone does not establish that no data leaves the machine.

# Overleaf, Statistical Software, and VS Code

## Statistical Software

Use the language extension or terminal workflow supported by your installation. Check paths, licenses, working directories, and the exact command on a small file.

## Overleaf

Agree where the authoritative draft lives and who synchronizes changes. Check available Git or GitHub integration for the account. Integrate reviewed changes through one owner, then inspect the compiled PDF.

Avoid concurrent edits to the same checkout through multiple sync systems. Keep a recoverable copy before synchronizing.

# The Makefile Approach

**A recipe for the project**

- Targets declare dependencies and commands
- Read a target before running it
- Inspect outputs after a successful build

In this repository, `make slides` first runs the analysis through `update-outputs`.

Back to Project Structure

**Commands for this revision**

```
make help
make slides-only
# Full research workflow:
make all
```

`slides-only` builds the Beamer deck from the included assets. It does not rerun the analysis.

# Research Exercise: Audit the Robustness Check

1. Open the existing paper's robustness table and its discussion.
2. Trace the table to `scripts/analysis.py`: default OLS versus standard errors clustered by `region_id`.
3. Ask one agent to check the implementation and another to compare the table with the prose.
4. Require file/line evidence, assumptions about clustering, and a list of claims that need a rerun to verify.
5. Draft a bounded issue and review plan before changing the analysis.

The original clustered-SE task is already implemented. Audit the existing work, or choose an instructor-approved extension. This optional exercise can replace the README exercise after rehearsal.

# VS Code Session Discovery: Scope Matters

- Local external Claude Code and Codex sessions can be discovered. They are hidden by default; enable *External* in the sessions-list filter.
- The setting is `chat.agentSessions.showExternal`.
- Codex continuation requires a single writer: fully quit the other application, then retry in VS Code.
- The OpenAI extension and experimental Codex Agent Host are distinct integration surfaces.

Remote SSH/tmux session discovery is not established by the local-session documentation. Rehearse the exact setup before promising it in a demo.

Manage sessions; agent harnesses.

# Resources

**Project Workflow**

- Git documentation
- GitHub documentation
- VS Code documentation
- GitHub Skills

**Agent Documentation**

- Claude Code
- Codex
- Model Context Protocol
- Workshop repository

Product availability depends on version, surface, account, and organization settings. The accompanying source notes record the documentation used for this revision.